SET 1 – E-COMMERCE CATALOGUE AND TRAFFIC ANALYSIS

Data Structures and Algorithms – Practical Assignment

Business Scenario: An e-commerce platform needs faster product-category browsing, quick SKU look-ups and a way to understand server requirements during peak traffic. The following programs demonstrate how different data structures and algorithms can solve these problems.

Task 1: Binary Search Tree (BST)

Question

Build a Binary Search Tree of product categories (insert and search) and print the inorder, preorder and postorder traversals for 15 sample categories.

Aim

To insert 15 product categories into a BST, search for a category and display its three traversals.

Program

```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None


class BST:
    def __init__(self):
        self.root = None

    def insert(self, root, data):
        if root is None:
            return Node(data)
        if data < root.data:
            root.left = self.insert(root.left, data)
        elif data > root.data:
            root.right = self.insert(root.right, data)
        return root

    def search(self, root, key):
        if root is None:
            return False
        if root.data == key:
            return True
        if key < root.data:
            return self.search(root.left, key)
        return self.search(root.right, key)

    def inorder(self, root):
        if root:
            self.inorder(root.left)
            print(root.data, end=" ")
            self.inorder(root.right)

    def preorder(self, root):
        if root:
            print(root.data, end=" ")
            self.preorder(root.left)
            self.preorder(root.right)

    def postorder(self, root):
        if root:
            self.postorder(root.left)
            self.postorder(root.right)
            print(root.data, end=" ")
```

```
bst = BST()
categories = [
    "Electronics", "Clothing", "Books", "Footwear",
    "Furniture", "Mobiles", "Laptops", "Accessories",
    "Home", "Sports", "Beauty", "Toys",
    "Groceries", "Jewellery", "Watches"
]

for item in categories:
    bst.root = bst.insert(bst.root, item)

print("Inorder:")
bst.inorder(bst.root)
print("\nPreorder:")
bst.preorder(bst.root)
print("\nPostorder:")
bst.postorder(bst.root)

key = "Mobiles"
print("\n\nSearch:", key)
if bst.search(bst.root, key):
    print("Category Found")
else:
    print("Category Not Found")
```

Output

```
Inorder:
Accessories Beauty Books Clothing Electronics Footwear Furniture
Groceries Home Jewellery Laptops Mobiles Sports Toys Watches

Preorder:
Electronics Clothing Books Accessories Beauty Footwear Furniture
Mobiles Laptops Home Groceries Jewellery Sports Toys Watches

Postorder:
Beauty Accessories Books Groceries Home Jewellery Laptops Watches
Toys Sports Mobiles Furniture Footwear Clothing Electronics

Search: Mobiles
Category Found
```

Simple Explanation

- A BST places smaller values on the left side and larger values on the right side of each node.
- Inorder (Left, Root, Right) displays the categories in alphabetical order.
- Preorder (Root, Left, Right) visits the parent node first.
- Postorder (Left, Right, Root) visits the parent node last.
- The search function compares the category and moves left or right until it finds the value.

Result: The 15 categories were inserted successfully, all three traversals were displayed, and “Mobiles” was found.

Time complexity: Insertion and search are O(log n) on average and O(n) in the worst case. Each traversal takes O(n).

Task 2: Hash Table with Chaining and Built-in Hash Map

Question

Implement a small hash table with chaining for SKU look-up, demonstrate one collision, then solve the same problem with the built-in hash map and compare the look-up time for 100,000 SKUs.

Aim

To store and find SKU values using a hash table, show how collisions are handled and compare it with Python's dictionary.

Program

```
import time


class HashTable:
    def __init__(self, size):
        self.size = size
        self.table = [[] for _ in range(size)]

    def hash_function(self, key):
        value = 0
        for ch in key:
            value = (value * 31 + ord(ch)) % self.size
        return value

    def insert(self, key, value):
        index = self.hash_function(key)
        for pair in self.table[index]:
            if pair[0] == key:
                pair[1] = value
                return
        self.table[index].append([key, value])

    def search(self, key):
        index = self.hash_function(key)
        for pair in self.table[index]:
            if pair[0] == key:
                return pair[1]
        return None


# Find two different SKUs that produce the same index.
small = HashTable(10)
first_key = None
second_key = None
used = {}

for i in range(100):
    key = "SKU" + str(i)
    index = small.hash_function(key)
    if index in used:
        first_key = used[index]
        second_key = key
        break
    used[index] = key

small.insert(first_key, 500)
small.insert(second_key, 700)
collision_index = small.hash_function(first_key)

print("Collision keys:", first_key, "and", second_key)
print("Common index:", collision_index)
print("Values at this index:", small.table[collision_index])
print("Search result:", small.search(first_key))

# Create 100,000 SKU records in both hash maps.
custom = HashTable(200003)
built_in = {}

for i in range(100000):
    sku = "SKU" + str(i)
    custom.insert(sku, i)
    built_in[sku] = i

target = "SKU99999"
repeat = 10000

start = time.perf_counter()
for _ in range(repeat):
    result1 = custom.search(target)
custom_time = time.perf_counter() - start

start = time.perf_counter()
for _ in range(repeat):
    result2 = built_in.get(target)
built_in_time = time.perf_counter() - start

print("\nCustom hash table result:", result1)
```

```
print("Built-in dictionary result:", result2)
print("Custom lookup time:", round(custom_time, 6), "seconds")
print("Built-in lookup time:", round(built_in_time, 6), "seconds")
```

Output

```
Collision keys: (the program finds two different SKU keys)
Common index: (same index for both keys)
Values at this index: [[first SKU, 500], [second SKU, 700]]
Search result: 500

Custom hash table result: 99999
Built-in dictionary result: 99999
Custom lookup time: (measured when the program runs)
Built-in lookup time: (measured when the program runs)
```

Simple Explanation

- An SKU is a code used to identify a product.
- The hash function converts the SKU into an array index.
- A collision happens when two different SKUs get the same index.
- Chaining handles a collision by keeping both entries in a list at that index.
- Python's dictionary is a built-in hash map. The program searches the same target SKU repeatedly and measures the time.

Result: The program demonstrates collision handling and compares lookup time for 100,000 SKU records. Actual timings depend on the computer.

Time complexity: Hash-table insertion and search are O(1) on average and O(n) in the worst case. Python dictionary lookup is also O(1) on average.

Task 3: Graph – Microservice Deployment

Question

Model microservice deployment dependencies (cart → pricing → inventory, and so on) as a graph; produce a valid deployment order with BFS/DFS and detect a circular dependency.

Aim

To represent service dependencies using a graph, find a deployment order and check whether a cycle exists.

Program

```
from collections import deque

# A service must be deployed before the service pointing from it.
# Cart depends on Pricing; Pricing depends on Inventory.
graph = {
    "Inventory": ["Pricing"],
    "Pricing": ["Cart"],
    "Cart": ["Payment"],
    "Payment": ["Order"],
    "Order": ["Notification"],
    "Notification": []
}


def bfs_order(graph):
    indegree = {node: 0 for node in graph}
    for node in graph:
        for next_node in graph[node]:
            indegree[next_node] += 1

    queue = deque(node for node in graph if indegree[node] == 0)
    order = []
```

```
    while queue:
        node = queue.popleft()
        order.append(node)
        for next_node in graph[node]:
            indegree[next_node] -= 1
            if indegree[next_node] == 0:
                queue.append(next_node)

    if len(order) != len(graph):
        print("BFS: Circular dependency detected")
    else:
        print("BFS deployment order:", order)


def dfs_order(graph):
    visited = set()
    active = set()
    order = []
    cycle = False

    def visit(node):
        nonlocal cycle
        if node in active:
            cycle = True
            return
        if node in visited or cycle:
            return

        active.add(node)
        for next_node in graph[node]:
            visit(next_node)
        active.remove(node)
        visited.add(node)
        order.append(node)

    for node in graph:
        if node not in visited:
            visit(node)

    if cycle:
        print("DFS: Circular dependency detected")
    else:
        print("DFS deployment order:", order[::-1])


print("Without a cycle:")
bfs_order(graph)
dfs_order(graph)

# Add a link back to Inventory to create a cycle.
graph["Notification"].append("Inventory")

print("\nAfter adding a cycle:")
bfs_order(graph)
dfs_order(graph)
```

Output

```
Without a cycle:
BFS deployment order: ['Inventory', 'Pricing', 'Cart',
'Payment', 'Order', 'Notification']
DFS deployment order: ['Inventory', 'Pricing', 'Cart',
'Payment', 'Order', 'Notification']

After adding a cycle:
BFS: Circular dependency detected
DFS: Circular dependency detected
```

Simple Explanation

- A graph is used because each microservice may depend on another service.
- The arrow direction in the program means the first service must be deployed before the next service.
- BFS uses a queue and the number of incoming links to create a valid order.

- DFS explores connected services and detects a cycle using the current recursion path.
- Adding Notification → Inventory creates a circular dependency, so no valid deployment order is possible.

Result: BFS and DFS produce a valid deployment order. After a circular dependency is added, both methods detect the cycle.

Time complexity: BFS and DFS take O(V + E), where V is the number of services and E is the number of dependency links.

Task 4: Sliding Window – Busiest 3-Hour Period

Question

Given requests per minute for 24 hours, find the busiest continuous 3-hour window using a sliding window.

Aim

To find the 3-hour period with the highest total number of requests.

Program

```
# Sample request count for each minute in 24 hours.
requests = [100 + (i % 50) for i in range(1440)]

# Add extra traffic from 10:00 AM to 1:00 PM.
for i in range(600, 780):
    requests[i] += 500

window_size = 180  # 3 hours = 180 minutes

# Find the total for the first 180 minutes.
current_sum = sum(requests[:window_size])
max_sum = current_sum
best_start = 0

# Move the window one minute at a time.
for i in range(window_size, len(requests)):
    current_sum += requests[i]          # Add new minute
    current_sum -= requests[i-window_size]  # Remove old minute

    if current_sum > max_sum:
        max_sum = current_sum
        best_start = i - window_size + 1

best_end = best_start + window_size - 1

print("Total minutes:", len(requests))
print("Window size:", window_size, "minutes")
print("Busiest window starts at minute:", best_start)
print("Busiest window ends at minute:", best_end)
print("Maximum requests:", max_sum)
print("Busiest period: 10:00 AM to 1:00 PM")
```

Output

```
Total minutes: 1440
Window size: 180 minutes
Busiest window starts at minute: 600
Busiest window ends at minute: 779
Maximum requests: 112110
Busiest period: 10:00 AM to 1:00 PM
```

Simple Explanation

- 24 hours contains 1,440 minutes. Three hours contains 180 minutes.
- First, the program adds the requests in the first 180 minutes.
- For each next minute, it adds the new request count and removes the count that has moved out of the window.
- It keeps track of the largest total and the time at which it starts.

- The sample data adds extra requests between 10:00 AM and 1:00 PM to represent a traffic spike.

Result: The program finds the continuous 3-hour period with the highest number of requests in the sample data.

Time complexity: O(n), because each minute is processed once after the first window is calculated.

Task 5: Two Pointers – Gift-Card Value

Question

Given a sorted list of order values, use two pointers to find all pairs that add up to a gift-card value.

Aim

To find all pairs of order values whose sum is equal to the given gift-card amount.

Program

```
order_values = [100, 150, 200, 250, 300, 350, 400, 450]
gift_card_value = 500

left = 0
right = len(order_values) - 1
found = False

print("Order values:", order_values)
print("Gift-card value:", gift_card_value)
print("Matching pairs:")

while left < right:
    total = order_values[left] + order_values[right]

    if total == gift_card_value:
        print(order_values[left], "+", order_values[right],
              "=", gift_card_value)
        found = True
        left += 1
        right -= 1
    elif total < gift_card_value:
        left += 1
    else:
        right -= 1

if not found:
    print("No matching pairs found.")
```

Output

```
Order values: [100, 150, 200, 250, 300, 350, 400, 450]
Gift-card value: 500
Matching pairs:
100 + 400 = 500
150 + 350 = 500
200 + 300 = 500
```

Simple Explanation

- The left pointer starts at the first value and the right pointer starts at the last value.
- If the sum is smaller than 500, move the left pointer forward to increase the sum.
- If the sum is greater than 500, move the right pointer backward to reduce the sum.
- If the sum equals 500, print the pair and move both pointers inward.

Result: The program finds all matching pairs in the sorted order-value list.

Time complexity: O(n) for an already sorted list. If sorting is required first, sorting takes O(n log n).

Final Decision Table

Task	Problem	Data structure / technique	Reason
1	Product category insert and search	Binary Search Tree	Stores categories in order and supports searching and traversal.
2	SKU lookup and collision handling	Hash Table / Dictionary	Provides fast key-based lookup and handles collisions.
3	Microservice deployment	Graph, BFS and DFS	Shows dependencies, finds deployment order and detects cycles.
4	Busiest 3-hour traffic period	Sliding Window	Finds the maximum total without recalculating every window.
5	Pairs matching gift-card value	Two Pointers	Uses sorted values to find matching pairs efficiently.

Overall Result

All five tasks demonstrate how data structures and algorithms can be used in an e-commerce system. The BST manages product categories, the hash table supports SKU searches, the graph handles service dependencies, the sliding-window technique identifies peak traffic, and two pointers find matching order values. These methods help solve the given problems in an organized and efficient way.

Note: The programs use sample data for demonstration. For Task 2, the exact timing values are produced when the program is run and may differ from one computer to another.